SENIOR PROJECT · SYSTEM OVERVIEW

AI-Powered Intelligent Document Processing and Administrative Automation Assistant

Karam Masry · September 2026

AGENDA

What This Covers

The problem and objectives

System architecture and tech stack

The AI pipeline — YOLO, PaddleOCR, Ollama

Two trained models — exams and invoices, kept separate

Live camera recognition

User accounts & access control

Email Manager — Gmail + local AI, fully live

The Email Canvas — a 2D board for conversations

Engineering challenges solved

Current status and what's next

MOTIVATION

The Problem

Small and medium enterprises lose real time to repetitive administrative work:

- Manually typing invoice data into spreadsheets
- Reading, sorting, and replying to routine email
- Summarizing long documents by hand

Constraint from day one: everything runs on local, offline AI — no per-document cloud API costs, and sensitive business/student data never has to leave the machine.

PROJECT PLAN

Objectives

1. Analyze administrative bottlenecks in SME workflows (invoicing, email handling)
2. Design and build a web application that automates these tasks end to end
3. Integrate AI for detection (YOLO), text extraction (PaddleOCR), understanding (Ollama), and email (Gmail API)
4. Evaluate real performance: accuracy, time saved, error reduction vs. manual work

DESIGN

System Architecture

React Frontend (Vite · TailwindCSS · React Router)

REST API — HTTP calls to /api/...

FastAPI Backend (Python · SQLite · bcrypt + token sessions)

↓

AI Pipeline — YOLOv8 → PaddleOCR (documents) · Ollama (email intelligence) · Gmail API (mail in/out)

WHAT IT'S BUILT ON

Technology Stack & Resources

Layer	Tools
Frontend	React 19, Vite, TailwindCSS v4, React Router, lucide-react
Backend	FastAPI, SQLite, Pydantic, uv (isolated Python 3.12 env)
Auth	bcrypt password hashing, opaque bearer-token sessions, role-based access
Detection	Ultralytics YOLOv8 (two fine-tuned models, ≥50% confidence, per-class caps)
OCR	PaddleOCR 3.7 / PaddlePaddle (CPU inference)
Language model	Ollama, local (qwen3.5:4b, CPU) — classification, summarization, drafting
Email	Gmail API, OAuth2 (read, send, modify), custom 2D canvas UI (no charting library)
Labeling	make-sense.ai — self-hosted from source
Documents	PyMuPDF (PDF rendering), Typst (this deck + the written report)

WHAT'S BUILT

Frontend

A full, authenticated application — every page wired to real backend data:

Dashboard

Documents
upload · camera · live detect

Invoices
same, invoice-tuned

Email Manager
list + 2D canvas views

Voice Assistant

Settings

Login

User Management
admin-only

Plus a persisted dark/light theme across the whole app.

WHAT'S BUILT

Backend

- FastAPI application with a full SQLite schema: documents, invoices, emails, attachments, sender names, summaries, voice commands, users, sessions, settings
- `/api/system/status` — genuinely checks (not assumes) whether YOLO, PaddleOCR, Ollama, and voice are live
- Session-based auth: every route gated behind a bearer token, admin-only routes enforced server-side
- Background-task processing so uploads respond instantly while detection/OCR run behind the scenes
- Schema evolves in place — new columns/tables are added via safe in-app migrations, existing data is never dropped

HOW A DOCUMENT GETS PROCESSED

The AI Pipeline

1. Upload (PDF or image) → rendered to a raster image if needed
2. **YOLO** proposes bounding boxes for the regions it recognizes
3. **PaddleOCR** reads the whole page for a general transcript
4. PaddleOCR runs **again on each individual region** YOLO detected

Result: text comes back already tied to its meaning — `{"subject": "Calculus II"}`, not an anonymous line in a wall of text.

DETECTION

Two Specialized Models, Kept Separate

exams.pt — 10 classes	**invoices.pt** — 16 classes
lu_logo, question, subject, date, faculty, duration, semester, department, university_name, instructions	invoice number, invoice date, total, header, Products, Tax, Subtotal, Discount, billing/shipping address, client contact, ...

Each page (Documents vs. Invoices) explicitly loads the right model. Detections below **50% confidence** are dropped, and single-occurrence exam fields (logo, duration, subject) are capped to their real-world count — the highest-confidence box wins if the model reports duplicates.

LABELING WORKFLOW

Building the Exam Dataset

Real invoices are hard to collect in bulk as a student. Lebanese University exam papers were a genuinely useful stand-in — structurally similar (header, logo, boxed fields) and available in quantity.

- Labeled with **make-sense.ai, self-hosted** — exam images carry student names/IDs, so nothing goes to a third-party server
- Grew from **23 → 128** labeled images across departments as more became available
- Two annotation batches arrived in different formats (YOLO `.txt`, Pascal VOC `.xml`) — unified with a custom converter

lu-exam	buscomp	civileng	comms	indeng
23	44	61	0	0

VALIDATION

First Training Run — Honest Results

YOLOv8n fine-tuned on 128 images, 90/19/19 split, CPU-only: 73 epochs, roughly 12 minutes.

- Overall mAP50 ≈ **0.68** — but the class with real sample size tells the true story
- `question` (9 real val instances): **0.89 mAP50** — genuinely strong
- Verified on a **held-out** exam page the model never saw: correctly located and read out a real multi-part question about Dijkstra's algorithm

Two thinnest-box classes underperformed with only about 20 examples each — expected, and the clear next lever is more labeled data, not a different architecture.

FEATURE

Live Camera Recognition

- Point the camera at a document — boxes track detected fields in **near real time**
- Each frame is also OCR'd (rate-limited to the top few boxes, mirroring reference desktop-script behavior)
- OCR's job here is **not** to display text live — it validates the box: a green outline means the reading matches what that field should look like (a real date in a date box); orange means it doesn't

Same mechanism, two independent models — Documents uses the exam model, Invoices uses the invoice model.

SECURITY

User Accounts & Access Control

- Passwords hashed with **bcrypt** — never stored or logged in plain text
- Login issues an opaque bearer token (7-day session), not a JWT — nothing about the user is decodable client-side
- Every API route is gated behind authentication by default; admin-only routes (user management) are enforced **server-side**, not just hidden in the UI
- A **User Management** page lets an admin create accounts, change roles, and remove users — with guardrails against locking yourself out (can't demote/delete the last admin, or yourself)

GMAIL + OLLAMA

Email Manager — Local AI, Fully Live

- Gmail API (OAuth2) fetches and sends mail — the one piece that necessarily talks to the cloud
- **Everything else stays local**: Ollama classifies (urgent / follow-up / action / info / spam), summarizes threads, and drafts replies — optionally steered with custom instructions
- **Classify All** runs Ollama across every unclassified email in one pass
- Sending is **gated behind explicit review** — a Gmail-style compose window (From/To/Subject shown, body editable) opens for every reply, AI-drafted or fully manual, and nothing goes out until Send is clicked
- Mark read/unread, delete-to-trash with one-click undo, attachment download, and per-sender custom nicknames — all synced back to Gmail, not just the local cache

A 2D BOARD FOR CONVERSATIONS

The Email Canvas

Instead of a flat inbox list, email lives on an **infinite pannable, zoomable board**:

- **Bordered regions** group email by AI-assigned category (Urgent, Action, Follow-up, Info, Spam) — each its own zone on the canvas
- **Threads branch visually**: a reply exchange draws as connected nodes with a smooth curved line, colored by category — a real back-and-forth is instantly readable as a chain, not scattered rows
- Reclassifying an email **snaps its card into the correct region automatically**; dragging a card is clamped to stay inside its own category's box
- Card positions are **saved to the server per email** — the layout you build persists across reloads and re-syncs

Every card is a full workspace on its own — classify, summarize, draft, reply, mark read

THE UNGLAMOROUS HALF

Engineering Challenges Solved

- System Python (3.14) too new for PaddlePaddle → isolated Python 3.12 via uv
- PyTorch's default install pulls roughly 2GB of unused CUDA libs on this GPU-less machine → CPU-only wheel
- PaddlePaddle 3.3.1 CPU inference crashed (oneDNN bug) → traced to a specific env flag fix
- PaddleOCR 3.x's real API differs completely from older documented examples → rewritten against what's actually installed
- Gmail OAuth silently failed ("missing code verifier") → real PKCE state-handling bug, found and fixed
- A reasoning-capable Ollama model generated 3000+ tokens of hidden "thinking" for a five-word reply (7+ minutes) → traced via the server's own timing log, fixed with a single API flag
- Adding a new Gmail permission scope silently broke the **existing** connection on token refresh → caught and degraded gracefully to "reconnect", instead of a server error

WHERE THINGS STAND

Current Status vs. Roadmap

Area	Status
Frontend (8 views, auth-gated, dark theme)	Done
Backend + SQLite schema	Done
User accounts, roles, admin management	Done
YOLO + PaddleOCR pipeline, both domains	Done
Exam model trained & validated	Done — scaling dataset
Invoice model integrated	Done — needs real-world testing
Camera capture + live recognition	Done
Gmail + Ollama email pipeline	Done — live end to end
Email Canvas (2D board, branches, positions)	Done
Voice input (VoxType)	Not started

ROADMAP

What's Next

1. Label a larger, department-balanced batch (comms, indeng) and retrain on GPU hardware
2. Test the invoice model against real invoice photos, not synthetic test images
3. Extend Ollama beyond email into document **field structuring** — today's document extraction is region + regex based, not LLM-assisted
4. Begin voice input (VoxType) — intentionally last, since it depends on the document pipeline being solid first

Thank You

Questions welcome.